

# Chapter 22 — Binary Search Tree

## 0. What You Will Learn in This Chapter

A Binary Search Tree, or BST, is a special binary tree where values follow an ordering rule.

For every node:

all values in left subtree  $<$  node value  $<$  all values in right subtree

This ordering makes BST problems different from normal binary tree problems.

By the end of this chapter, you should understand:

- What a BST is
- Why BST property matters
- How inorder traversal gives sorted order
- How to search in a BST
- How to insert into a BST
- How to delete from a BST
- How to validate a BST
- How to find kth smallest
- How to find lowest common ancestor in a BST
- How to find closest value
- How to recover swapped BST
- How BST connects to binary search
- How BST differs from heap
- When BST operations are  $O(\log n)$  and when they degrade to  $O(n)$

Core problems covered in this chapter:

1. Search in a Binary Search Tree
  2. Insert into a Binary Search Tree
  3. Validate Binary Search Tree
  4. Kth Smallest Element in a BST
  5. Lowest Common Ancestor of a BST
  6. Minimum Absolute Difference in BST
  7. Convert Sorted Array to BST
  8. Delete Node in a BST
  9. Range Sum of BST
  10. Trim a Binary Search Tree
  11. Closest Binary Search Tree Value
  12. Recover Binary Search Tree
  13. Two Sum IV — Input is a BST
  14. BST Iterator
  15. Balance a Binary Search Tree
  16. Convert BST to Greater Tree
  17. Construct BST from Preorder Traversal
  18. Unique Binary Search Trees, DP preview
- 

## 1. What Is a Binary Search Tree?

A Binary Search Tree is a binary tree with an ordering rule.

For every node:

left subtree values are smaller  
right subtree values are larger

Example:

```
  5
 /\
3  7
 /\  \
2  4  8
```

This is a valid BST.

For node 5:

left subtree: 2, 3, 4 all  $< 5$   
right subtree: 7, 8 all  $> 5$

For node 3:

left subtree:  $2 < 3$   
right subtree:  $4 > 3$

## 2. Why BST Exists

BST exists to support ordered operations efficiently.

In a balanced BST, search works like binary search.

At each node:

- if target equals current value, found
- if target is smaller, go left
- if target is larger, go right

This eliminates half-ish of the remaining tree when balanced.

Average balanced time:

$O(\log n)$

Worst case skewed tree:

$O(n)$

## 3. Core BST Property

The main property is:

left subtree  $<$  root  $<$  right subtree

Important:

This applies to the **entire subtree**, not just direct children.

Invalid example:

```
  5
 /\
1  7
```

/ \  
4 8

Node 4 is in the right subtree of 5, but  $4 < 5$ .  
So this is not a valid BST.

## 4. BST vs Normal Binary Tree

| Structure         | any shape           | ordered by value    |
|-------------------|---------------------|---------------------|
| Search            | $O(n)$              | $O(h)$              |
| Inorder traversal | arbitrary order     | sorted order        |
| LCA               | general recursion   | use value direction |
| Kth smallest      | traversal all nodes | inorder sorted      |
| Range queries     | scan all nodes      | prune branches      |

## 5. BST vs Heap

BST and heap are often confused.

| Main property          | $\text{left} < \text{root} < \text{right}$ | parent priority over children |
|------------------------|--------------------------------------------|-------------------------------|
| Search arbitrary value | $O(\log n)$ if balanced                    | $O(n)$                        |
| Get min/max            | leftmost/rightmost                         | top only                      |
| Inorder sorted?        | yes                                        | no                            |
| Shape                  | can be unbalanced                          | usually complete tree         |
| Used for               | ordered search                             | priority queue                |

A heap is not a BST.

## 6. When to Use BST Property

Use BST-specific logic when the problem says:

- binary search tree
- sorted tree
- kth smallest
- range sum
- closest value
- LCA in BST

- validate BST
- insert/delete/search in BST

If problem only says binary tree, do not assume BST ordering.

---

## 7. When NOT to Use BST Logic

### 7.1 Tree Is Not Guaranteed BST

If the problem says binary tree, use general tree DFS/BFS.

### 7.2 BST Is Skewed

Operations may be  $O(n)$ , not  $O(\log n)$ .

Example:

```

1
 \
 2
  \
  3
   \
    4
  
```

### 7.3 Need Level Order

Use BFS.

### 7.4 Need Subtree Aggregation

Use Tree DFS.

BST property helps with ordering, not all tree computations.

---

## 8. Mental Models

### 8.1 Binary Search on Tree

At each node:

```

target smaller -> go left
target larger -> go right
  
```

### 8.2 Inorder Means Sorted

For BST:

```

left -> root -> right
  
```

produces sorted values.

### 8.3 Bounds Model

Each node has valid range:

```

low < node value < high
  
```

Left child tightens high.

Right child tightens low.

### 8.4 Pruning Model

For range queries:

If node value is too small, skip left subtree.  
If node value is too large, skip right subtree.

---

## 9. Pattern Recognition Checklist

Ask:

19. Does the problem explicitly say BST?
20. Can I use  $\text{left} < \text{root} < \text{right}$ ?
21. Does inorder traversal help?
22. Is sorted order useful?
23. Can I eliminate one subtree based on value?
24. Is the problem asking kth smallest/largest?
25. Is it a range query?
26. Is it LCA in a BST?
27. Is insertion/deletion/search required?
28. Do I need validate using bounds?

If yes, use BST property.

---

## 10. C++ Tools Needed

### 10.1 TreeNode

```
struct TreeNode {  
    int val;  
    TreeNode* left;  
    TreeNode* right;  
  
    TreeNode() : val(0), left(nullptr), right(nullptr) {}  
  
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}  
  
    TreeNode(int x, TreeNode* left, TreeNode* right)  
        : val(x), left(left), right(right) {}  
};
```

### 10.2 Long Long Bounds

For validation:

```
LLONG_MIN  
LLONG_MAX
```

Use long long because node value may be INT\_MIN or INT\_MAX.

### 10.3 Inorder Vector

```
vector<int> values;
```

### 10.4 Stack for Iterative Inorder

```
stack<TreeNode*> st;
```

---

# 11. Core Templates

## Template 1 — Search in BST

```
TreeNode* searchBST(TreeNode* root, int val) {
    if (root == nullptr || root->val == val) {
        return root;
    }

    if (val < root->val) {
        return searchBST(root->left, val);
    }

    return searchBST(root->right, val);
}
```

## Template 2 — Iterative Search

```
TreeNode* searchBST(TreeNode* root, int val) {
    while (root != nullptr && root->val != val) {
        if (val < root->val) {
            root = root->left;
        } else {
            root = root->right;
        }
    }

    return root;
}
```

## Template 3 — BST Inorder

```
void inorder(TreeNode* root, vector<int>& values) {
    if (root == nullptr) {
        return;
    }

    inorder(root->left, values);
    values.push_back(root->val);
    inorder(root->right, values);
}
```

## Template 4 — Validate BST With Bounds

```
bool validate(TreeNode* root, long long low, long long high) {
    if (root == nullptr) {
        return true;
    }

    if (root->val <= low || root->val >= high) {
        return false;
    }

    return validate(root->left, low, root->val) &&
        validate(root->right, root->val, high);
}
```

## Template 5 — Insert Into BST

```
TreeNode* insertIntoBST(TreeNode* root, int val) {
    if (root == nullptr) {
        return new TreeNode(val);
    }

    if (val < root->val) {
        root->left = insertIntoBST(root->left, val);
    } else {
        root->right = insertIntoBST(root->right, val);
    }

    return root;
}
```

## Template 6 — Kth Smallest Inorder

```
void inorder(TreeNode* root, int k, int& count, int& answer) {
    if (root == nullptr) {
        return;
    }

    inorder(root->left, k, count, answer);

    count++;

    if (count == k) {
```

```

        answer = root->val;
        return;
    }

    inorder(root->right, k, count, answer);
}

```

## Template 7 — LCA in BST

```

TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root != nullptr) {
        if (p->val < root->val && q->val < root->val) {
            root = root->left;
        } else if (p->val > root->val && q->val > root->val) {
            root = root->right;
        } else {
            return root;
        }
    }

    return nullptr;
}

```

## Template 8 — Range Pruning

```

int rangeSumBST(TreeNode* root, int low, int high) {
    if (root == nullptr) {
        return 0;
    }

    if (root->val < low) {
        return rangeSumBST(root->right, low, high);
    }

    if (root->val > high) {
        return rangeSumBST(root->left, low, high);
    }

    return root->val +
        rangeSumBST(root->left, low, high) +
        rangeSumBST(root->right, low, high);
}

```

---

## 12. Major BST Variations

|                       |                       |                     |
|-----------------------|-----------------------|---------------------|
|                       |                       |                     |
| Search                | choose left/right     | Search BST          |
| Insert                | recurse to null spot  | Insert BST          |
| Delete                | handle 0/1/2 children | Delete Node         |
| Validate              | bounds                | Validate BST        |
| Kth smallest          | inorder sorted        | Kth Smallest        |
| Range sum             | prune subtrees        | Range Sum BST       |
| LCA                   | split point by values | LCA BST             |
| Closest value         | binary search path    | Closest BST Value   |
| Recover swapped BST   | inorder anomaly       | Recover BST         |
| Iterator              | controlled inorder    | BST Iterator        |
| Balanced construction | sorted array middle   | Sorted Array to BST |

## 13. Full Problem Walkthrough 1 — LeetCode 700: Search in a Binary Search Tree

### 13.1 Problem Summary

Given root of BST and value val, return subtree rooted at node with val.  
If not found, return nullptr.

---

### 13.2 How to Identify BST Pattern

Problem says BST and asks search.  
Use ordering property.

---

### 13.3 C++ Code

```
class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {
        if (root == nullptr || root->val == val) {
            return root;
        }
    }
};
```

```

    }

    if (val < root->val) {
        return searchBST(root->left, val);
    }

    return searchBST(root->right, val);
}
};

```

## 13.4 Iterative Version

```

class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {
        while (root != nullptr && root->val != val) {
            if (val < root->val) {
                root = root->left;
            } else {
                root = root->right;
            }
        }

        return root;
    }
};

```

## 13.5 Complexity

Time:

O(h)

Balanced:

O(log n)

Skewed:

O(n)

Space recursive:

O(h)

Space iterative:

O(1)

# 14. Full Problem Walkthrough 2 — LeetCode 701: Insert into a Binary Search Tree

## 14.1 Problem Summary

Insert value into BST and return root.

---

## 14.2 How to Identify BST Pattern

Insertion follows search path until null child.

---

## 14.3 C++ Code

```
class Solution {
public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        if (root == nullptr) {
            return new TreeNode(val);
        }

        if (val < root->val) {
            root->left = insertIntoBST(root->left, val);
        } else {
            root->right = insertIntoBST(root->right, val);
        }

        return root;
    }
};
```

## 14.4 Iterative Version

```
class Solution {
public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {

        TreeNode* curr = root;

        while (true) {
```

```
        if (val < curr->val) {
            if (curr->left == nullptr) {
                curr->left = new TreeNode(val);
                break;
            }

            curr = curr->left;
        } else {
            if (curr->right == nullptr) {
                curr->right = new TreeNode(val);
                break;
            }

            curr = curr->right;
        }
    }

    return root;
}
```

## 14.5 Complexity

Time:

$O(h)$

Space recursive:

$O(h)$

Space iterative:

$O(1)$

# 15. Full Problem Walkthrough 3 — LeetCode 98: Validate Binary Search Tree

## 15.1 Problem Summary

Return true if binary tree is valid BST.

## 15.2 How to Identify BST Pattern

Need verify global BST property.

Use recursive bounds.

---

## 15.3 C++ Code

```
class Solution {
public:
    bool validate(TreeNode* root, long long low, long long high) {
        if (root == nullptr) {
            return true;
        }

        if (root->val <= low || root->val >= high) {
            return false;
        }

        return validate(root->left, low, root->val) &&
            validate(root->right, root->val, high);
    }

    bool isValidBST(TreeNode* root) {
        return validate(root, LLONG_MIN, LLONG_MAX);
    }
};
```

## 15.4 Inorder Alternative

Inorder traversal of BST must be strictly increasing.

```
class Solution {
public:
    TreeNode* prev = nullptr;

    bool isValidBST(TreeNode* root) {
        if (root == nullptr) {
            return true;
        }

        if (!isValidBST(root->left)) {
            return false;
        }

        if (prev != nullptr && root->val <= prev->val) {
            return false;
        }
    }
};
```

```
    prev = root;

    return isValidBST(root->right);
}
};
```

## 15.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

## 15.6 Common Mistakes

- Only comparing direct children
- Allowing duplicates
- Using int bounds and failing at INT\_MIN / INT\_MAX

# 16. Full Problem Walkthrough 4 — LeetCode 230: Kth Smallest Element in a BST

## 16.1 Problem Summary

Return kth smallest element in BST.

## 16.2 How to Identify BST Pattern

BST inorder traversal gives sorted order.

Kth smallest is kth visited node in inorder.

## 16.3 C++ Code

```
class Solution {
public:
    int count = 0;
    int answer = -1;

    void inorder(TreeNode* root, int k) {
        if (root == nullptr) {
            return;
        }
    }
```

```

    inorder(root->left, k);

    count++;

    if (count == k) {
        answer = root->val;
        return;
    }

    inorder(root->right, k);
}

int kthSmallest(TreeNode* root, int k) {
    inorder(root, k);

    return answer;
}
};

```

## 16.4 Iterative Version

```

class Solution {
public:
    int kthSmallest(TreeNode* root, int k) {
        stack<TreeNode*> st;
        TreeNode* curr = root;

        while (curr != nullptr || !st.empty()) {
            while (curr != nullptr) {
                st.push(curr);
                curr = curr->left;
            }

            curr = st.top();
            st.pop();

            k--;

            if (k == 0) {
                return curr->val;
            }

            curr = curr->right;
        }
    }
};

```

```

    }

    return -1;
}
};

```

## 16.5 Complexity

Time:

$O(h + k)$

Space:

$O(h)$

# 17. Full Problem Walkthrough 5 — LeetCode 235: Lowest Common Ancestor of a BST

## 17.1 Problem Summary

Given BST and nodes p, q, return their lowest common ancestor.

## 17.2 How to Identify BST Pattern

In BST:

- if both values smaller than root, LCA is in left subtree
- if both larger, LCA is in right subtree
- otherwise root is split point and LCA

## 17.3 C++ Code

```

class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
    TreeNode* q) {
        while (root != nullptr) {
            if (p->val < root->val && q->val < root->val) {
                root = root->left;
            } else if (p->val > root->val && q->val > root->val) {
                root = root->right;
            } else {
                return root;
            }
        }
    }
}

```

```
        return nullptr;
    }
};
```

## 17.4 Complexity

Time:

$O(h)$

Space:

$O(1)$

# 18. Full Problem Walkthrough 6 — LeetCode 530: Minimum Absolute Difference in BST

## 18.1 Problem Summary

Given BST, return minimum absolute difference between values of any two nodes.

## 18.2 How to Identify BST Pattern

Inorder traversal gives sorted values.

Minimum difference in sorted list occurs between adjacent values.

## 18.3 C++ Code

```
class Solution {
public:
    TreeNode* prev = nullptr;
    int answer = INT_MAX;

    void inorder(TreeNode* root) {
        if (root == nullptr) {
            return;
        }

        inorder(root->left);

        if (prev != nullptr) {
            answer = min(answer, root->val - prev->val);
        }
    }
};
```

```
        prev = root;

        inorder(root->right);
    }

    int getMinimumDifference(TreeNode* root) {
        inorder(root);

        return answer;
    }
};
```

## 18.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

# 19. Full Problem Walkthrough 7 — LeetCode 108: Convert Sorted Array to Binary Search Tree

## 19.1 Problem Summary

Given sorted array, convert it to height-balanced BST.

## 19.2 How to Identify BST Pattern

Sorted array should become BST.

Choose middle as root for balance.

Left half builds left subtree.

Right half builds right subtree.

## 19.3 C++ Code

```
class Solution {
public:
    TreeNode* build(vector<int>& nums, int left, int right) {
        if (left > right) {
            return nullptr;
        }
    }
```

```
int mid = left + (right - left) / 2;

TreeNode* root = new TreeNode(nums[mid]);

root->left = build(nums, left, mid - 1);
root->right = build(nums, mid + 1, right);

return root;
}

TreeNode* sortedArrayToBST(vector<int>& nums) {
    return build(nums, 0, nums.size() - 1);
}
};
```

## 19.4 Complexity

Time:

$O(n)$

Space:

$O(\log n)$

for balanced recursion stack.

# 20. Full Problem Walkthrough 8 — LeetCode 450: Delete Node in a BST

## 20.1 Problem Summary

Delete node with given key from BST and return root.

## 20.2 How to Identify BST Pattern

Search for key using BST property.  
Then handle deletion cases.

## 20.3 Deletion Cases

### Case 1 — No Child

Return nullptr.

### Case 2 — One Child

Return that child.

### Case 3 — Two Children

Replace current value with inorder successor, which is smallest node in right subtree.  
Then delete successor from right subtree.

---

## 20.4 C++ Code

```
class Solution {
public:
    TreeNode* findMin(TreeNode* root) {
        while (root->left != nullptr) {
            root = root->left;
        }

        return root;
    }

    TreeNode* deleteNode(TreeNode* root, int key) {
        if (root == nullptr) {
            return nullptr;
        }

        if (key < root->val) {
            root->left = deleteNode(root->left, key);
        } else if (key > root->val) {
            root->right = deleteNode(root->right, key);
        } else {
            if (root->left == nullptr) {
                return root->right;
            }

            if (root->right == nullptr) {
                return root->left;
            }

            TreeNode* successor = findMin(root->right);

            root->val = successor->val;

            root->right = deleteNode(root->right, successor->val);
        }

        return root;
    }
};
```

```
}  
};
```

## 20.5 Complexity

Time:

$O(h)$

Space:

$O(h)$

## 20.6 Common Mistakes

- Not returning updated root
- Mishandling two-child case
- Forgetting to delete successor
- Breaking BST property

# 21. Full Problem Walkthrough 9 — LeetCode 938: Range Sum of BST

## 21.1 Problem Summary

Return sum of values in BST within range  $[low, high]$ .

## 21.2 How to Identify BST Pattern

Use BST property to prune.

If root value is too small, skip left.

If too large, skip right.

## 21.3 C++ Code

```
class Solution {  
public:  
    int rangeSumBST(TreeNode* root, int low, int high) {  
        if (root == nullptr) {  
            return 0;  
        }  
  
        if (root->val < low) {  
            return rangeSumBST(root->right, low, high);  
        }  
  
        if (root->val > high) {  
            return rangeSumBST(root->left, low, high);  
        }  
    }  
};
```

```
        return root->val +
            rangeSumBST(root->left, low, high) +
            rangeSumBST(root->right, low, high);
    }
};
```

## 21.4 Complexity

Average balanced:

$O(\log n + k)$

where k is number of nodes in range.

Worst case:

$O(n)$

# 22. Full Problem Walkthrough 10 — LeetCode 669: Trim a Binary Search Tree

## 22.1 Problem Summary

Given BST and range [low, high], trim tree so all values remain in range.

Return new root.

## 22.2 How to Identify BST Pattern

If root value is below low, entire left subtree is below root and invalid.

Return trimmed right.

If root value is above high, entire right subtree invalid.

Return trimmed left.

## 22.3 C++ Code

```
class Solution {
public:
    TreeNode* trimBST(TreeNode* root, int low, int high) {
        if (root == nullptr) {
            return nullptr;
        }

        if (root->val < low) {
            return trimBST(root->right, low, high);
        }
    }
};
```

```

    if (root->val > high) {
        return trimBST(root->left, low, high);
    }

    root->left = trimBST(root->left, low, high);
    root->right = trimBST(root->right, low, high);

    return root;
}
};

```

## 22.4 Complexity

Time:

$O(n)$

Worst case.

Balanced with pruning can be less.

Space:

$O(h)$

# 23. Full Problem Walkthrough 11 — Closest Binary Search Tree Value

## 23.1 Problem Summary

Given BST and target double, return value in BST closest to target.

## 23.2 How to Identify BST Pattern

Search path can be pruned using BST property.

At each node, update closest.

Then move left or right depending on target.

## 23.3 C++ Code

```

int closestValue(TreeNode* root, double target) {
    int closest = root->val;

    while (root != nullptr) {
        if (abs(root->val - target) < abs(closest - target)) {
            closest = root->val;
        }
    }
}

```

```
    if (target < root->val) {  
        root = root->left;  
    } else {  
        root = root->right;  
    }  
}  
  
return closest;  
}
```

## 23.4 Complexity

Time:

$O(h)$

Space:

$O(1)$

# 24. Full Problem Walkthrough 12 — LeetCode 99: Recover Binary Search Tree

## 24.1 Problem Summary

Two nodes in BST are swapped by mistake.

Recover tree without changing structure.

## 24.2 How to Identify BST Pattern

Inorder traversal of BST should be sorted.

Swapped nodes create inversions.

Example sorted should be:

1, 2, 3, 4

Swapped:

1, 4, 3, 2

Inversions:

$4 > 3$

$3 > 2$

First wrong node is 4.

Second wrong node is 2.

## 24.3 C++ Code

```

class Solution {
public:
    TreeNode* first = nullptr;
    TreeNode* second = nullptr;
    TreeNode* prev = nullptr;

    void inorder(TreeNode* root) {
        if (root == nullptr) {
            return;
        }

        inorder(root->left);

        if (prev != nullptr && prev->val > root->val) {
            if (first == nullptr) {
                first = prev;
            }

            second = root;
        }

        prev = root;

        inorder(root->right);
    }

    void recoverTree(TreeNode* root) {
        inorder(root);

        swap(first->val, second->val);
    }
};

```

## 24.4 Complexity

Time:

O(n)

Space:

O(h)

## 24.5 Common Mistakes

- Only handling adjacent swapped nodes

- Not updating second on second inversion
  - Swapping nodes instead of values unnecessarily
- 

## 25. Full Problem Walkthrough 13 — LeetCode 653: Two Sum IV — Input is a BST

### 25.1 Problem Summary

Given BST and integer k, return true if there are two nodes whose values sum to k.

---

### 25.2 Approach 1 — Inorder + Two Pointers

BST inorder gives sorted array.

Then use two pointers.

---

### 25.3 C++ Code

```
class Solution {
public:
    void inorder(TreeNode* root, vector<int>& values) {
        if (root == nullptr) {
            return;
        }

        inorder(root->left, values);
        values.push_back(root->val);
        inorder(root->right, values);
    }

    bool findTarget(TreeNode* root, int k) {
        vector<int> values;

        inorder(root, values);

        int left = 0;
        int right = values.size() - 1;

        while (left < right) {
            int sum = values[left] + values[right];

            if (sum == k) {
                return true;
            }
        }

        return false;
    }
};
```

```
        } else if (sum < k) {
            left++;
        } else {
            right--;
        }
    }

    return false;
}
```

## 25.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

## 25.5 Alternative

DFS + hash set also works in  $O(n)$  time.

# 26. Full Problem Walkthrough 14 — LeetCode 173: Binary Search Tree Iterator

## 26.1 Problem Summary

Implement BST iterator with:

```
next()
hasNext()
```

`next()` returns next smallest number.

## 26.2 How to Identify BST Pattern

Need controlled inorder traversal.

Use stack to simulate recursion.

## 26.3 C++ Code

```
class BSTIterator {
private:
    stack<TreeNode*> st;
```

```

void pushLeft(TreeNode* root) {
    while (root != nullptr) {
        st.push(root);
        root = root->left;
    }
}

public:
    BSTIterator(TreeNode* root) {
        pushLeft(root);
    }

    int next() {
        TreeNode* node = st.top();
        st.pop();

        if (node->right != nullptr) {
            pushLeft(node->right);
        }

        return node->val;
    }

    bool hasNext() {
        return !st.empty();
    }
};

```

## 26.4 Complexity

next() amortized:

$O(1)$

Space:

$O(h)$

Each node is pushed and popped once.

# 27. Full Problem Walkthrough 15 — LeetCode 1382: Balance a Binary Search Tree

## 27.1 Problem Summary

Given BST, return balanced BST with same values.

---

## 27.2 How to Identify BST Pattern

Inorder gives sorted values.

Build balanced BST from sorted array.

---

## 27.3 C++ Code

```
class Solution {
public:
    void inorder(TreeNode* root, vector<int>& values) {
        if (root == nullptr) {
            return;
        }

        inorder(root->left, values);
        values.push_back(root->val);
        inorder(root->right, values);
    }

    TreeNode* build(vector<int>& values, int left, int right) {
        if (left > right) {
            return nullptr;
        }

        int mid = left + (right - left) / 2;

        TreeNode* root = new TreeNode(values[mid]);

        root->left = build(values, left, mid - 1);
        root->right = build(values, mid + 1, right);

        return root;
    }

    TreeNode* balanceBST(TreeNode* root) {
```

```
vector<int> values;

inorder(root, values);

return build(values, 0, values.size() - 1);
}
};
```

## 27.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

# 28. Full Problem Walkthrough 16 — LeetCode 538: Convert BST to Greater Tree

## 28.1 Problem Summary

Convert BST so each node value becomes original value plus sum of all greater values.

## 28.2 How to Identify BST Pattern

Need visit values from largest to smallest.

Use reverse inorder:

right -> root -> left

## 28.3 C++ Code

```
class Solution {
public:
    int runningSum = 0;

    TreeNode* convertBST(TreeNode* root) {
        if (root == nullptr) {
            return nullptr;
        }

        convertBST(root->right);
```

```
        runningSum += root->val;
        root->val = runningSum;

        convertBST(root->left);

        return root;
    }
};
```

## 28.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

# 29. Full Problem Walkthrough 17 — LeetCode 1008: Construct BST from Preorder Traversal

## 29.1 Problem Summary

Given preorder traversal of BST, construct the tree.

## 29.2 How to Identify BST Pattern

Preorder visits:

root, left subtree, right subtree

BST gives bounds for valid values.

## 29.3 C++ Code

```
class Solution {
public:
    TreeNode* build(vector<int>& preorder, int& index, int bound) {
        if (index == preorder.size() || preorder[index] > bound) {
            return nullptr;
        }

        TreeNode* root = new TreeNode(preorder[index]);
        index++;
```

```

    root->left = build(preorder, index, root->val);
    root->right = build(preorder, index, bound);

    return root;
}

TreeNode* bstFromPreorder(vector<int>& preorder) {
    int index = 0;

    return build(preorder, index, INT_MAX);
}
};

```

## 29.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

# 30. Full Problem Walkthrough 18 — LeetCode 96: Unique Binary Search Trees

## 30.1 Problem Summary

Given  $n$ , return number of structurally unique BSTs storing values  $1..n$ .

## 30.2 How to Identify Pattern

Choose each value as root.

If root is  $i$ :

- left subtree has  $i - 1$  values
- right subtree has  $n - i$  values

Number of trees:

$dp[n] = \text{sum}(dp[i - 1] * dp[n - i])$

This is DP, but based on BST structure.

## 30.3 C++ Code

```

class Solution {
public:
    int numTrees(int n) {
        vector<int> dp(n + 1, 0);

        dp[0] = 1;
        dp[1] = 1;

        for (int nodes = 2; nodes <= n; nodes++) {
            for (int root = 1; root <= nodes; root++) {
                dp[nodes] += dp[root - 1] * dp[nodes - root];
            }
        }

        return dp[n];
    }
};

```

### 30.4 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

## 31. Mini Problem 1 — Find Minimum in BST

```

int findMin(TreeNode* root) {
    while (root->left != nullptr) {
        root = root->left;
    }

    return root->val;
}

```

## 32. Mini Problem 2 — Find Maximum in BST

```
int findMax(TreeNode* root) {  
    while (root->right != nullptr) {  
        root = root->right;  
    }  
  
    return root->val;  
}
```

## 33. Mini Problem 3 — Inorder Successor in BST

```
TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {  
    TreeNode* successor = nullptr;  
  
    while (root != nullptr) {  
        if (p->val < root->val) {  
            successor = root;  
            root = root->left;  
        } else {  
            root = root->right;  
        }  
    }  
  
    return successor;  
}
```

## 34. Mini Problem 4 — Inorder Predecessor in BST

```
TreeNode* inorderPredecessor(TreeNode* root, TreeNode* p) {  
    TreeNode* predecessor = nullptr;  
  
    while (root != nullptr) {  
        if (p->val > root->val) {  
            predecessor = root;  
            root = root->right;  
        } else {  
            root = root->left;  
        }  
    }  
}
```

```
}  
  
return predecessor;  
}
```

## 35. Common Edge Cases for BST

### 35.1 Empty Tree

Return depends on problem.

### 35.2 One Node

Search, insert, kth smallest, validation should all work.

### 35.3 Skewed BST

Time becomes  $O(n)$ .

### 35.4 Duplicate Values

Most LeetCode BST problems assume no duplicates or strict inequality.

Validate carefully.

### 35.5 INT\_MIN / INT\_MAX

Use long long bounds.

### 35.6 Deleting Root

Deletion must return updated root.

## 36. Common Mistakes and Wrong Code

### Mistake 1 — Only Checking Children for Validation

Wrong:

```
root->left->val < root->val
```

Need full bounds.

### Mistake 2 — Forgetting BST Can Be Skewed

Do not always claim  $O(\log n)$ .

Say  $O(h)$ , balanced  $O(\log n)$ , worst  $O(n)$ .

### Mistake 3 — Mishandling Duplicates

For strict BST:

```
left < root < right
```

not  $\leq$ .

### Mistake 4 — Kth Smallest Without Inorder

BST sorted property appears only in inorder.

## Mistake 5 — Delete Node Without Returning New Root

When deleting root, returned root may change.

---

# 37. Interview Explanation Templates

## 37.1 Search BST

At each node, I compare target with node value. If target is smaller, I search left; if larger, I search right. This uses the BST ordering property.

## 37.2 Validate BST

Each node must satisfy bounds inherited from ancestors. Left subtree values must be less than current node, and right subtree values greater. I recursively update bounds.

## 37.3 Kth Smallest

Inorder traversal of BST visits values in ascending order. I count visited nodes and return the kth value.

## 37.4 LCA in BST

If both target nodes are smaller than root, LCA is left. If both are larger, LCA is right. Otherwise, current root is the split point and is the LCA.

## 37.5 Delete BST Node

I first find the node using BST search. If it has zero or one child, return the child. If it has two children, replace its value with inorder successor and delete the successor.

---

# 38. Complexity Patterns

| Search BST          | $O(h)$       | $O(h)$ recursive / $O(1)$ iterative |
|---------------------|--------------|-------------------------------------|
| Insert BST          | $O(h)$       | $O(h)$ recursive / $O(1)$ iterative |
| Validate BST        | $O(n)$       | $O(h)$                              |
| Kth Smallest        | $O(h + k)$   | $O(h)$                              |
| LCA in BST          | $O(h)$       | $O(1)$ iterative                    |
| Minimum Difference  | $O(n)$       | $O(h)$                              |
| Sorted Array to BST | $O(n)$       | $O(\log n)$                         |
| Delete Node         | $O(h)$       | $O(h)$                              |
| Range Sum           | $O(n)$ worst | $O(h)$                              |

| Problem Type            | Time                  | Space  |
|-------------------------|-----------------------|--------|
| Trim BST                | $O(n)$ worst          | $O(h)$ |
| Closest Value           | $O(h)$                | $O(1)$ |
| Recover BST             | $O(n)$                | $O(h)$ |
| BST Iterator            | $O(1)$ amortized next | $O(h)$ |
| Balance BST             | $O(n)$                | $O(n)$ |
| Greater Tree            | $O(n)$                | $O(h)$ |
| Construct from Preorder | $O(n)$                | $O(h)$ |

## 39. Time-Boxed Practice Guidance

### Easy BST Problems

Target:

10–20 minutes

Examples:

- Search in BST
- Insert into BST
- Range Sum of BST
- Minimum Difference in BST

### Medium BST Problems

Target:

20–40 minutes

Examples:

- Validate BST
- Kth Smallest
- LCA of BST
- Delete Node
- Trim BST
- BST Iterator

### Harder BST Problems

Target:

40–70 minutes

Examples:

- Recover BST
- Balance BST
- Construct BST from Preorder
- Unique BSTs

- Serialize/Deserialize BST
- 

## 40. Expanded Practice Set

### 40.1 Must-Do Easy

29. Search in a Binary Search Tree
30. Insert into a Binary Search Tree
31. Range Sum of BST
32. Minimum Absolute Difference in BST
33. Convert Sorted Array to Binary Search Tree
34. Increasing Order Search Tree

### 40.2 Must-Do Medium

35. Validate Binary Search Tree
36. Kth Smallest Element in a BST
37. Lowest Common Ancestor of a BST
38. Delete Node in a BST
39. Trim a Binary Search Tree
40. Two Sum IV — Input is a BST
41. Binary Search Tree Iterator
42. Closest Binary Search Tree Value
43. Convert BST to Greater Tree

### 40.3 Strong Interview Practice

44. Recover Binary Search Tree
45. Balance a Binary Search Tree
46. Construct Binary Search Tree from Preorder Traversal
47. Serialize and Deserialize BST
48. Unique Binary Search Trees
49. Unique Binary Search Trees II
50. Inorder Successor in BST
51. Inorder Predecessor in BST

### 40.4 Hard / Stretch

52. Recover Binary Search Tree
  53. Unique Binary Search Trees II
  54. Count of Smaller Numbers After Self, BST approach
  55. Number of Ways to Reorder Array to Get Same BST
  56. Serialize and Deserialize BST
- 

## 41. Suggested Study Order

### Step 1 — Basic BST Navigation

Start with:

57. Search in BST
58. Insert into BST
59. Range Sum of BST

Goal:

Use left/right direction decisions.

---

## Step 2 — Inorder Property

Solve:

- 60. Kth Smallest
- 61. Minimum Absolute Difference
- 62. Two Sum IV
- 63. BST Iterator

Goal:

Understand inorder sorted order.

---

## Step 3 — Validation and Bounds

Solve:

- 64. Validate BST
- 65. Trim BST
- 66. Closest BST Value

Goal:

Use ancestor constraints and pruning.

---

## Step 4 — Structural Modification

Solve:

- 67. Delete Node in BST
- 68. Convert Sorted Array to BST
- 69. Balance BST

Goal:

Build and modify BST safely.

---

## Step 5 — Advanced BST

Solve:

- 70. Recover BST
- 71. Construct BST from Preorder
- 72. Unique BSTs

Goal:

Handle advanced inorder and recursive construction.

---

# 42. Pattern Decision Table

| find value             | binary search path   | Search BST     |
|------------------------|----------------------|----------------|
| insert value           | recurse to null spot | Insert BST     |
| validate tree          | bounds               | Validate BST   |
| kth smallest           | inorder count        | Kth Smallest   |
| sorted order           | inorder traversal    | Min Difference |
| lowest common ancestor | split point          | LCA BST        |

| Problem Clue          | BST Technique         | Example             |
|-----------------------|-----------------------|---------------------|
| range query           | prune left/right      | Range Sum           |
| trim by range         | return valid subtree  | Trim BST            |
| closest value         | search path           | Closest BST Value   |
| delete key            | successor replacement | Delete Node         |
| iterator              | stack inorder         | BST Iterator        |
| recover swapped nodes | inorder inversion     | Recover BST         |
| sorted array input    | choose middle         | Sorted Array to BST |
| preorder input        | bounds recursion      | Construct BST       |

## 43. Final Cheat Sheet

### Search

```

while (root != nullptr) {
    if (root->val == val) {
        return root;
    }

    if (val < root->val) {
        root = root->left;
    } else {
        root = root->right;
    }
}

return nullptr;

```

### Inorder

```

inorder(root->left);
process(root);
inorder(root->right);

```

### Validate Bounds

```

bool validate(TreeNode* root, long long low, long long high) {
    if (root == nullptr) {
        return true;
    }

```

```

if (root->val <= low || root->val >= high) {
    return false;
}

return validate(root->left, low, root->val) &&
    validate(root->right, root->val, high);
}

```

## Kth Smallest

```

inorder left
count++
if count == k answer = root->val
inorder right

```

## LCA BST

```

if (p->val < root->val && q->val < root->val) {
    root = root->left;
} else if (p->val > root->val && q->val > root->val) {
    root = root->right;
} else {
    return root;
}

```

## Range Pruning

```

if (root->val < low) {
    return dfs(root->right);
}

if (root->val > high) {
    return dfs(root->left);
}

```

# 44. What to Master Before Chapter 23

Before moving to Ordered Set / Ordered Map, make sure you can:

- Explain BST property
- Search in BST
- Insert into BST
- Use inorder traversal for sorted order
- Validate BST with bounds
- Find kth smallest with inorder

- Find LCA in BST
  - Use range pruning
  - Delete node from BST
  - Find inorder successor/predecessor
  - Understand balanced vs skewed complexity
  - Know difference between BST and heap
  - Use iterative inorder with stack
  - Solve BST Iterator
  - Recover swapped BST using inorder anomalies
- 

## 45. Core Takeaway

A BST is a binary tree with ordered structure.

The key property is:

left subtree < root < right subtree

This gives two powerful tools:

### 73. **Directional search**

smaller -> left

larger -> right

### 74. **Sorted inorder traversal**

left -> root -> right

Most BST problems are built from these two ideas.

Always remember complexity depends on height:

$O(h)$

Balanced BST:

$O(\log n)$

Skewed BST:

$O(n)$

BST problems become straightforward when you ask:

Can I use the ordering property to skip work?